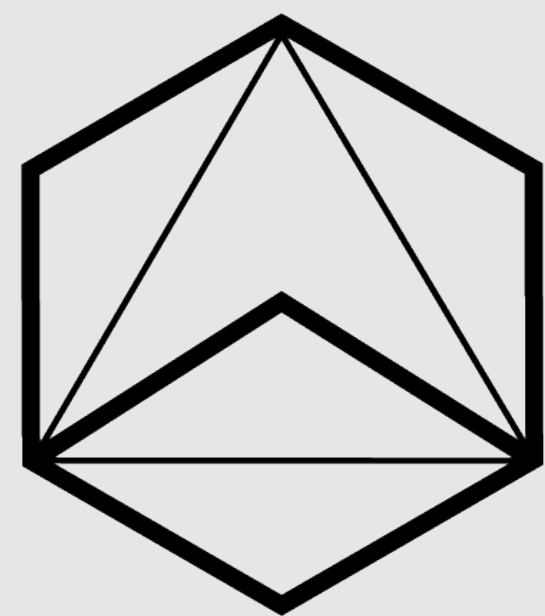




THE
AUTOWARE
FOUNDATION

Safety Island

Daniel SHIH

國立臺灣大學



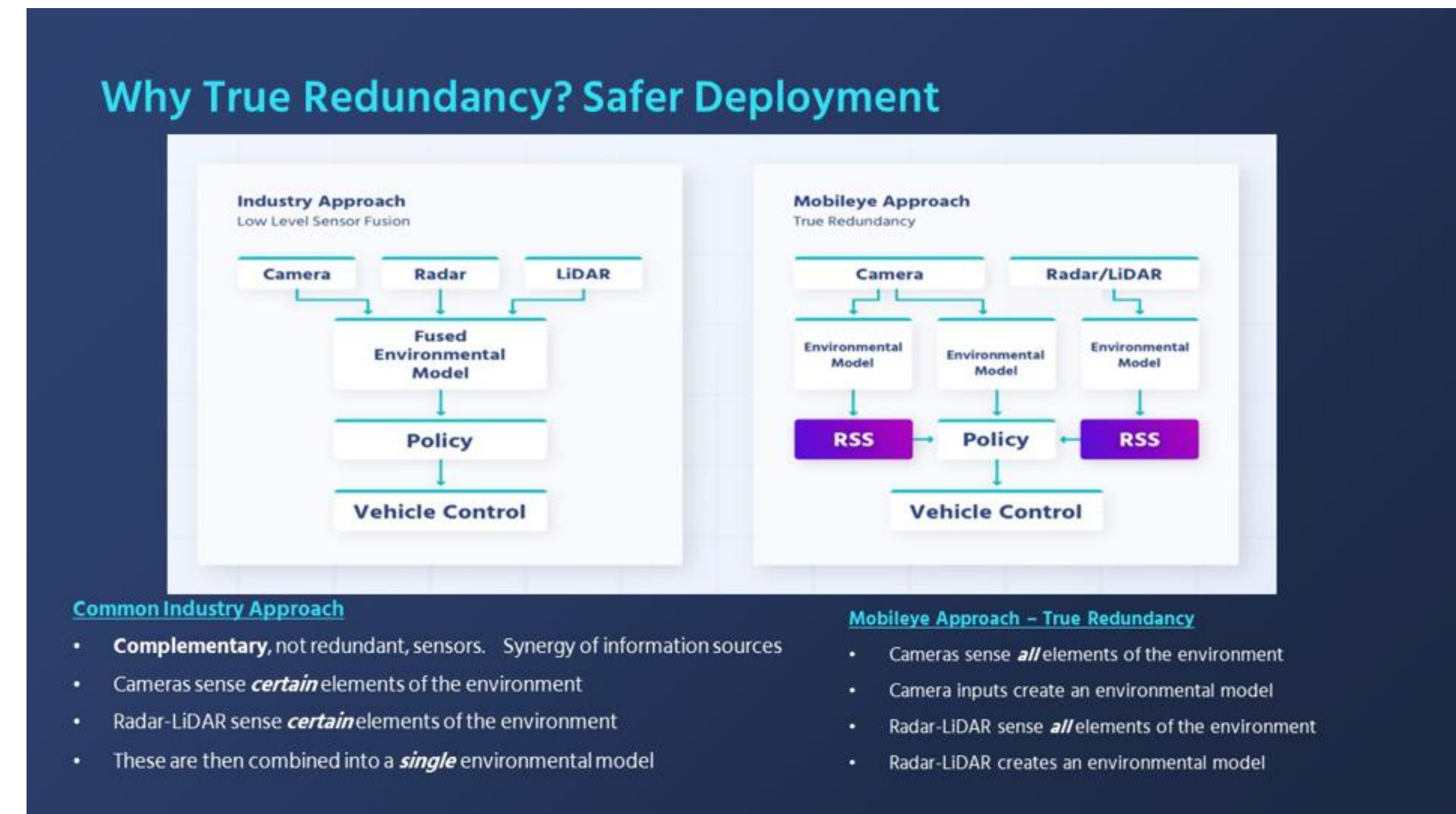
Author and Date

Agenda

- ▶ Definition of Safety Island: Role, Responsibilities, and Requirements.
- ▶ Architecture for Safety Island: Where should the Safety Island be deployed?
- ▶ Safety Island for L2 ADAS
- ▶ Safety Island for L4 Autonomous Driving

Safety at different levels

- ▶ **Functional Safety** by Redundancy/
Multiple-Versions:
 - ▶ Use redundant sensors/software/
hardware to ensure the
correctness of the functions.
- ▶ **Safety monitoring** by independent
SoC or computers:
 - ▶ **Monitor** the operations and **react**
when the operations fail to meet
the requirements to ensure the
system remains safe.



Safety Island - Detection

- ▶ The safety island should detect faults or unsafe conditions across four categories:
 - ▶ **Autonomy degradation:** perception confidence collapse, localization inconsistency, planning timeout, sensor disagreement, or anomalous neural-network outputs.
 - ▶ **Vehicle-control hazards:** excessive yaw rate, unintended acceleration, steering mismatch, braking-system fault, or trajectory deviation.
 - ▶ **Operational-design-domain violations:** poor weather, roadworks, unmapped areas, geofence breaches, or visibility conditions outside approved limits.
 - ▶ **Cyber and compute faults:** communication loss, abnormal resource usage, corrupted messages, watchdog timeout, or unauthorized command patterns.

Safety Island - Reaction

- ▶ When the primary stack becomes unreliable, the island should apply a graded response:
 - ▶ **Notify the primary stack** and request recovery or a conservative maneuver.
 - ▶ **Restrict speed** and enlarge safety margins.
 - ▶ **Take authority for a minimal-risk maneuver**, such as controlled deceleration, lane stabilization, hazard-light activation, and stopping at a safe location when feasible.
 - ▶ **Record the event** and report a privacy-preserving safety summary for audit.

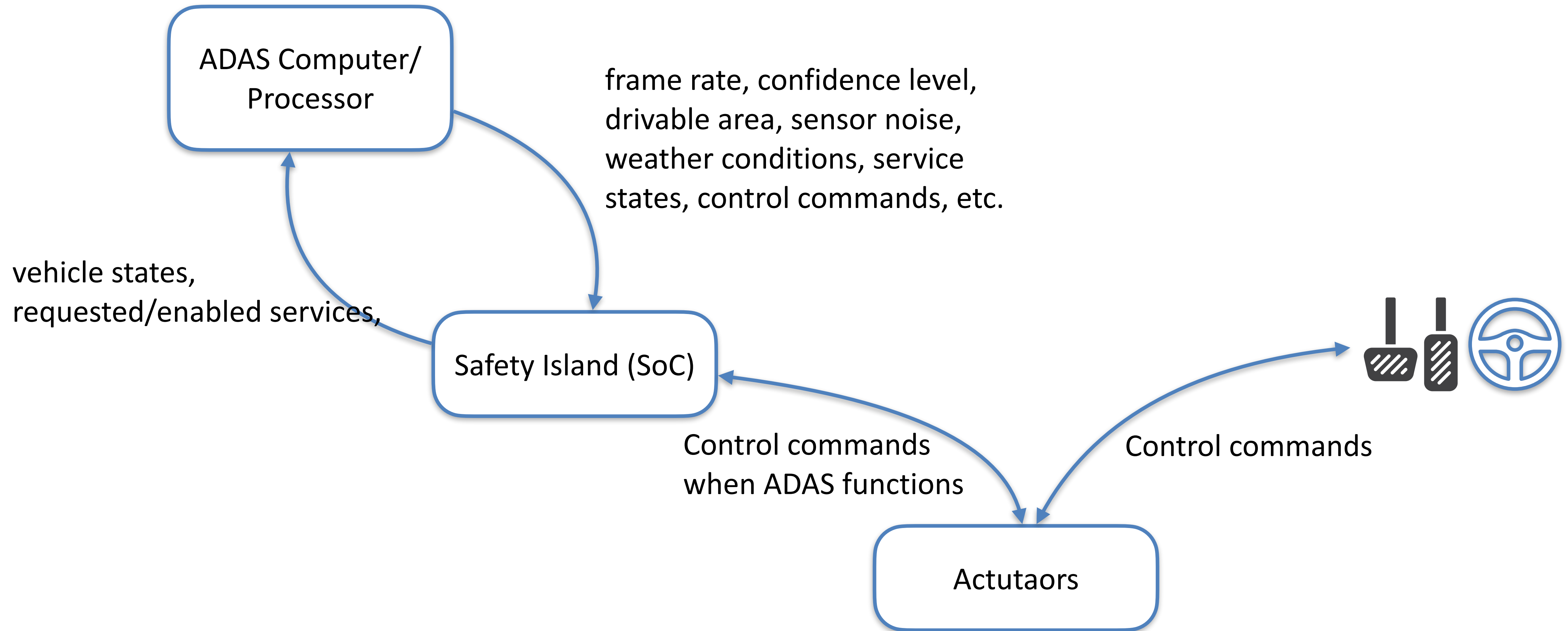
Safety Island

- ▶ A **safety island** is understood as an independent, safety-certified supervisory subsystem that monitors the main ADAS/automated-driving stack, detects dangerous faults or degraded performance, and either constrains the system, transfers control to the driver, or executes a minimal-risk maneuver.
- ▶ Due to differences in vehicle control ownership, the safety island for L2 and L4 will be designed accordingly.

Safety Island for L2 ADAS

- ▶ For most production Level 1–2 ADAS, the human driver remains the fallback.
- ▶ For example, a lane-centering system should disengage and alert the driver if camera visibility is lost, lane confidence becomes too low, steering actuator feedback is inconsistent, or the ADAS ECU fails timing diagnostics. The safety island's job is to ensure that a stale, corrupted, or implausible steering command cannot continue unchecked.

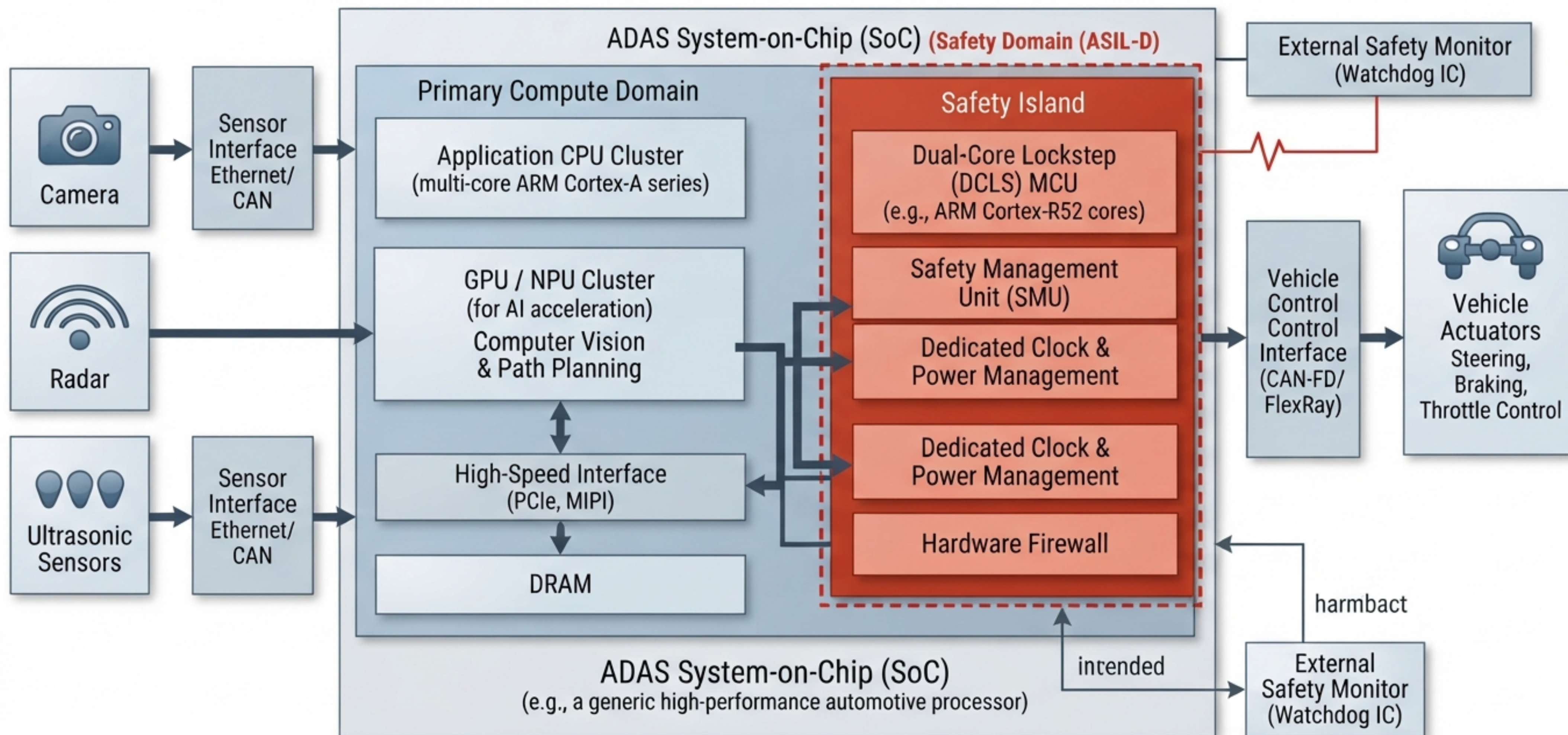
Safety Island for L2 ADAS



Architecture of Safety Island for L2 ADAS

- ▶ In Level 2 (L2) Advanced Driver Assistance Systems (ADAS), the safety island is deployed directly inside the System-on-Chip (SoC), typically integrated within the heterogeneous compute platform that runs the primary ADAS applications.
- ▶ Within the layered architecture of an L2 ADAS vehicle, the safety island sits at the hardware-firmware boundary of the primary processing unit.
- ▶ The safety island acts as the trusted supervisor between the untrusted high-compute software and the physical vehicle controls.

System Architecture: L2 ADAS with Integrated Safety Island



Middleware options for L2 Safety Island

Middleware / framework	Communication model	Safety-island suitability	Typical role
Zero-copy IPC: iceoryx	Shared-memory publish/subscribe on one SoC	Useful inside an HPC, but not the only safety boundary	Primary compute internal communication
Zenoh	Pub/sub-query distributed middleware	Promising for flexible integration; verify safety evidence	Research integration, gateway and mixed-network cases
AUTOSAR Classic RTE + COM	Signal-oriented, statically configured	Strong choice for MCU-based ASIL functions	Brake/steer gateway, vehicle-state signals, watchdog/fault state
AUTOSAR Adaptive ara::com	Service-oriented proxy/skeleton API	Suitable for HPC ↔ safety-domain interfaces when qualified	Primary ADAS command, health status, mode management
SOME/IP + Service Discovery	Service-oriented RPC, events, fields over Ethernet	Common automotive transport; must add E2E safety protection	Structured ADAS services and ECU-to-ECU events
DDS	Data-centric publish/subscribe with QoS	Well suited to decoupled ADAS and safety-supervision data flows	Trajectories, object lists, health monitoring, simulator/prototype integration
Eclipse Cyclone DDS / Fast DDS	Open-source DDS implementations	Good for research; qualification burden remains with integrator	ROS 2 / Autoware prototyping and non-final safety interfaces
RTI Connex Cert / Drive	Safety-certifiable or safety-certified DDS product family	Strong production candidate where DDS is selected	ASIL safety communication across ECUs/SoCs

Recommend to Autoware Vision Pilot

Updated on 0814.

- ▶ On L2 ADAS system, the safety island is implemented on integrated SoCs and Autoware Vision Pilot to be compliant with the selected Safety Island SoCs.
- ▶ Option 1: Follow the AUTOSAR protocol to interact with a third-party Safety Island SoC.
- ▶ Option 2: Use IPC or Zenoh to interact with an in-house safety island on the SoC.
 - ▶ Given the design of Autoware Vision Pilot, ROS message wrapper is available. Option 2 can use Zenoh-pico or nano-ros as the middleware to host the safety island features.
- ▶ The L2 safety island for Autoware should have the following features:
 - ▶ Monitoring the status of Autoware Vision Pilot, sensors, and weather conditions to determine whether the ASAD feature functions normally.
 - ▶ Assuring that only the correct control commands are sent to the actuators.

Communication for L2 Safety Island

In-vehicle communication

- ▶ Use CAN FD for compact safety control/status messages and Automotive Ethernet for high-rate sensor or trajectory data.
- ▶ Use IEEE 802.1AS/gPTP to establish a common time base; cross-sensor time alignment is necessary before claiming sensor disagreement or calculating accurate end-to-end latency.
- ▶ Put **AUTOSAR E2E protection** on every safety-relevant message, with message-specific timeout, counter window, CRC, and data-ID configuration

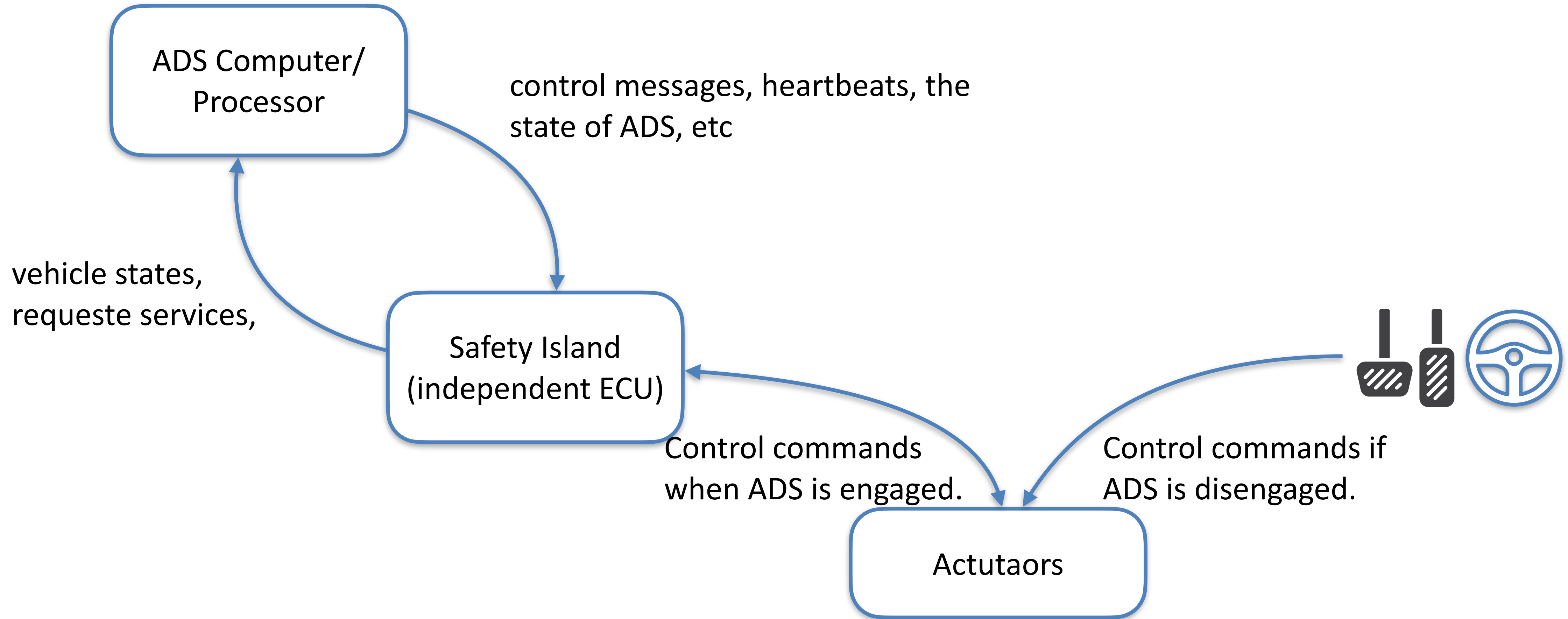
Interface purpose	Common protocol or standard	Safety relevance
Low-/medium-bandwidth control	CAN / CAN FD, typically ISO 11898 family	Robust distributed ECU control, status, diagnostics
Deterministic high-bandwidth ADAS backbone	Automotive Ethernet, often with TSN	Sensor/compute connectivity with bounded timing
Shared time base	IEEE 802.1AS / gPTP, AUTOSAR time synchronization	Align sensor, vehicle-state, and command timestamps
Safety payload integrity	AUTOSAR E2E protection	Detects corruption, repetition, loss, delay, insertion, and wrong addressing
ECU diagnostics	UDS, ISO 14229	Reads DTCs, diagnostic status, routine tests, safe reset/recovery workflows
UDS over Ethernet	ISO 14229-5 (UDSonIP)	Diagnostic access over IP/Ethernet
Sensor-to-fusion semantics	ISO 23150	Logical, modular, semantic sensor/fusion data interface
Predictive-map data	ADASIS v2	Horizon messages such as position, segment, profile, and metadata

Safety Island for L4 ADS

- ▶ **L2 safety island:** supervise an assistance feature and fail safely back to a capable driver.
- ▶ **L4 safety island:** supervise an autonomous vehicle and remain capable of safely controlling or stopping it after loss of the primary autonomy function.
- ▶ When L4 within ODD, ADS remains responsible and continue with degraded capability or execute an automated fallback to a minimal-risk condition.
- ▶ For an L3/L4 system, “turn everything off” is often unsafe—particularly at highway speed or in dense traffic. The fallback therefore needs independent capability to maintain lane position, control longitudinal motion, select a safe trajectory, activate warning lamps, and stop or otherwise reach a minimal-risk condition.

Safety Island for L4 ADS

Updated on 0814.



How should Autoware Interact with Safety Island

- ▶ Use distinct protocols for three different data classes rather than putting all traffic onto one ROS 2 or Ethernet channel.

Interface	Recommended protocol	Data type	Design rule
Sensor → primary ADS	Automotive Ethernet, SerDes, PCIe	High-bandwidth raw camera/lidar/radar data	Not the only source for fallback safety
Primary ADS → safety island	Automotive Ethernet with DDS or SOME/IP	Trajectory, command, health, mode, timestamp	Narrow, typed, rate-limited interface
On-SoC FSI → external MCU	SPI + discrete error GPIO; optionally isolated CAN FD	Hardware faults, watchdog, heartbeat, failover event	Avoid dependence on Linux/network switch
Safety MCU → actuator gateway	CAN FD or deterministic Automotive Ethernet	Brake/steer/propulsion limits, MRM command, actuator feedback	Independent safety route and E2E protection
Diagnostics / workshop / logging	UDS over CAN or UDS on IP	DTCs, health status, event evidence	Not part of timing-critical control loop
Network timing	IEEE 802.1AS/gPTP	Common synchronized time base	Needed for freshness and latency evidence

Middleware options for Safety Island

Middleware / framework	Communication model	Safety-island suitability	Typical role
Zero-copy IPC: iceoryx	Shared-memory publish/subscribe on one SoC	Useful inside an HPC, but not the only safety boundary	Primary compute internal communication
Zenoh	Pub/sub-query distributed middleware	Promising for flexible integration; verify safety evidence	Research integration, gateway and mixed-network cases
AUTOSAR Classic RTE + COM	Signal-oriented, statically configured	Strong choice for MCU-based ASIL functions	Brake/steer gateway, vehicle-state signals, watchdog/fault state
AUTOSAR Adaptive ara::com	Service-oriented proxy/skeleton API	Suitable for HPC ↔ safety-domain interfaces when qualified	Primary ADAS command, health status, mode management
SOME/IP + Service Discovery	Service-oriented RPC, events, fields over Ethernet	Common automotive transport; must add E2E safety protection	Structured ADAS services and ECU-to-ECU events
DDS	Data-centric publish/subscribe with QoS	Well suited to decoupled ADAS and safety-supervision data flows	Trajectories, object lists, health monitoring, simulator/prototype integration
Eclipse Cyclone DDS / Fast DDS	Open-source DDS implementations	Good for research; qualification burden remains with integrator	ROS 2 / Autoware prototyping and non-final safety interfaces
RTI Connex Cert / Drive	Safety-certifiable or safety-certified DDS product family	Strong production candidate where DDS is selected	ASIL safety communication across ECUs/SoCs

L4 Safety Island for Autoware

Updated on 0814.

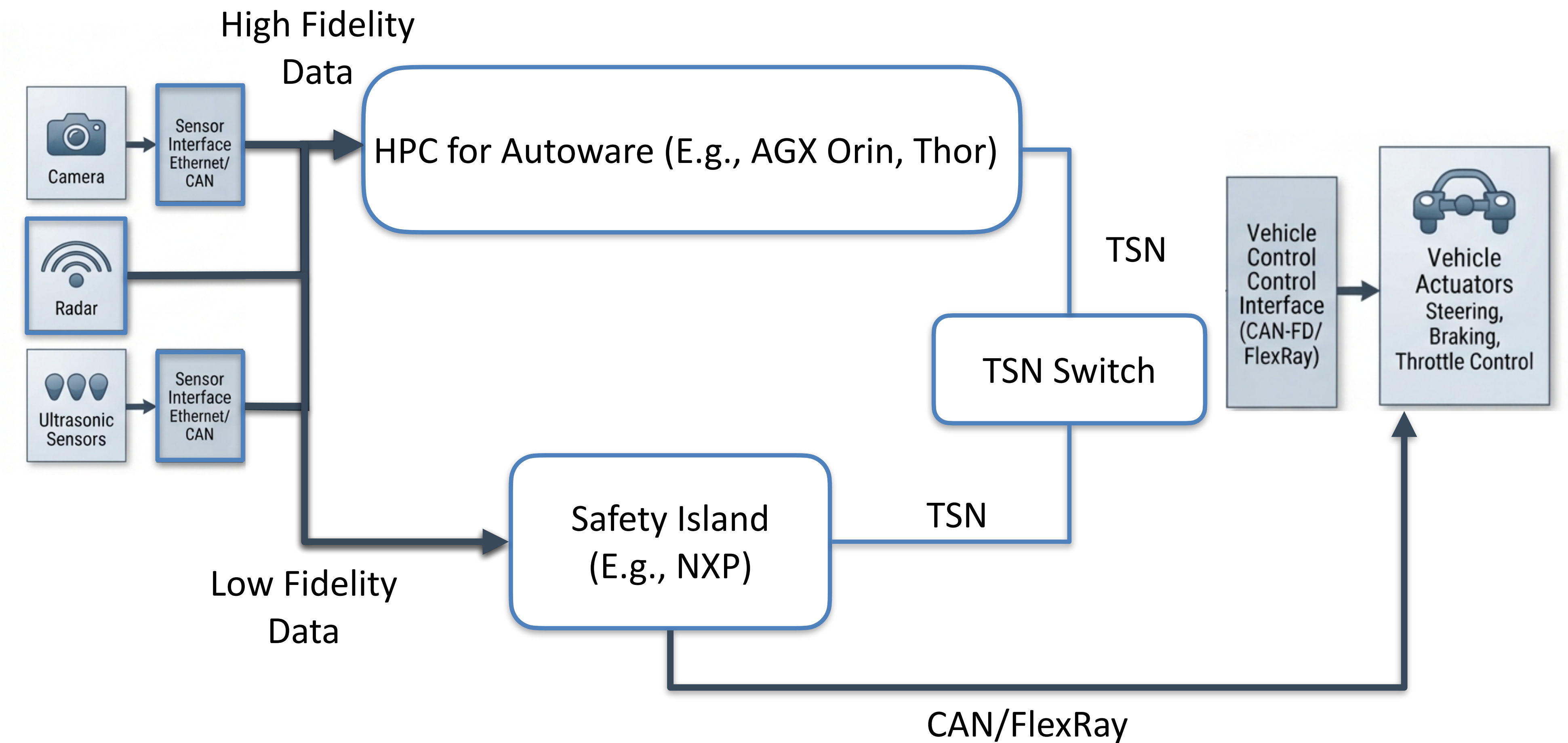
- ▶ The safety island will be part of Autoware, but implemented on a standalone ECU.
- ▶ On HPC, perception/planning/VLM are deployed.
- ▶ On the Safety Island, status monitoring, the control module (to convert the desired speed into control commands), and MRM are deployed.
- ▶ These ECUs should be connected by TSN/Vehicle Ethernet.
- ▶ The safety island must provide real-time guarantees for all the operations.
- ▶ Operating systems and autoware ros-node **on the safety island** must be real-time compliant: predictable execution times and memory safe.

Features on L4 Safety Islands

- ▶ Monitoring the sensor status and system states, including connectivity, availability, frame rates, weather conditions, confidence level of the perception, and ODD validity.
- ▶ Actions to keep the vehicles within the ODD, including converting the planned actions into control commands, falling back to minimal risk maneuvering, and disengaging the autonomous driving mode.

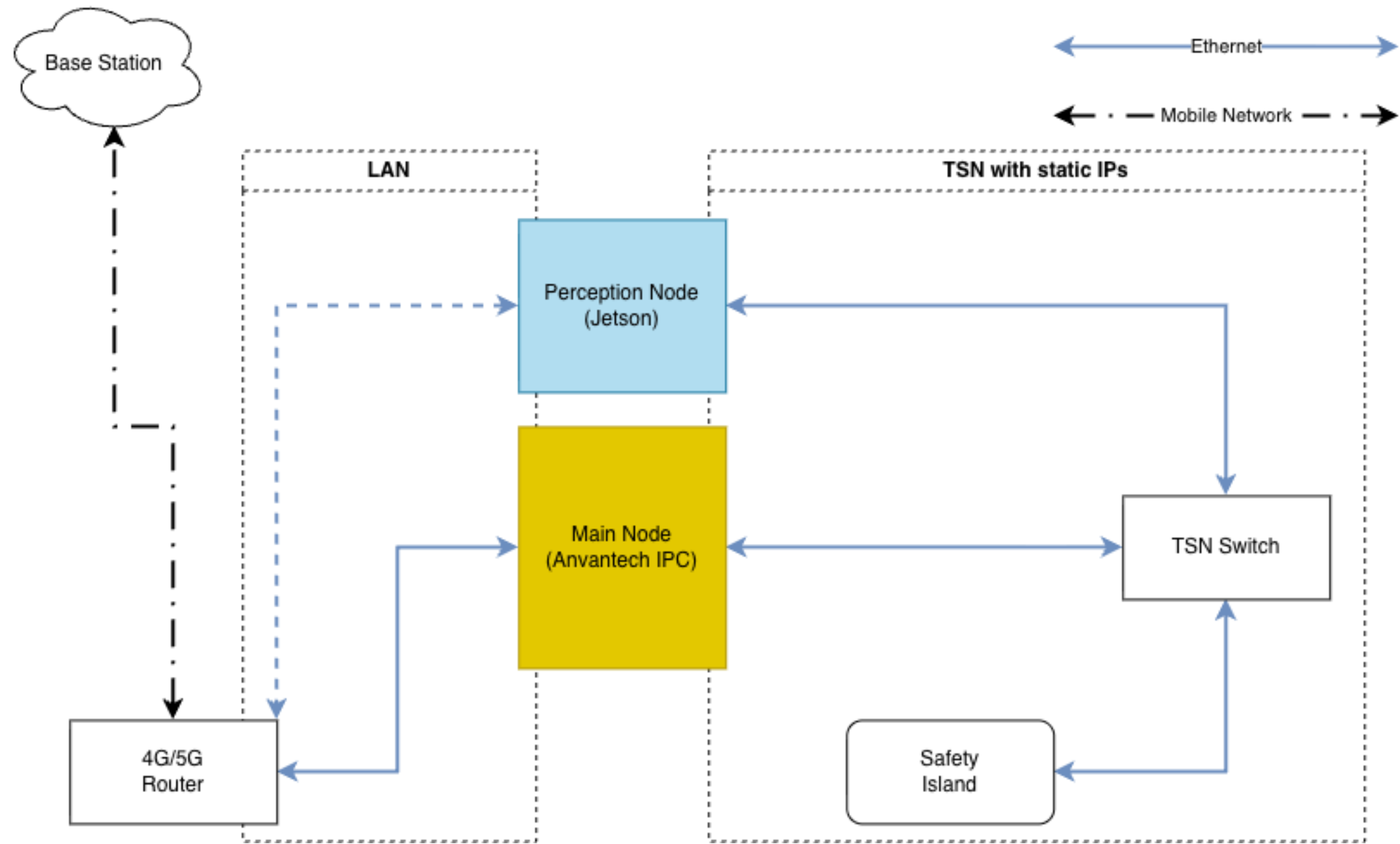
L4 Safety Island for Autoware (cont.)

Updated on 0814.



-
- The Orin Architecture diagram illustrates a complex system-on-chip (SoC) design. At the top, the **Safety Service** (CPU Power Rail) contains the **QNX RTOS** and **Trusted OS**, which interface with a **Hypervisor**. The **QNX BSP** (Clock & Reset, Timers, Mailboxes, Semaphore, Config Reg., Sys Config., GPIO Controller) and **QNX#BSP** (I2C, SPI, DMIC, GPIO, UART, I2C, QSPI, SPI, Fuses, PWFM, Safety I2S, USB, UFS, SDIMMC) are also shown. A **Sensor Processing MCU (R5)** is connected to the **QNX#BSP**. The **System Control Backbone** is a central hub connecting these components to the **System MMIO & Hypervisor** at the bottom. Various accelerators and components are connected to the backbone, including **BPMP** (Power Management), **NvMedia** (General Purpose), **CUDA & Tensor RT** (Tensor Cores), **Autosar/MicroSAR** (Safety), **NvStreams(& NvGPU)** (Safety), **PK CS #**, **NvMedia P/A**, **NvMedia D/A**, **CSI Camera Input**, **Camera ISP**, **Video Decoder**, **NvMedia Image Processor**, **NvDisplay**, and **General purpose DMA**. The **System MMIO & Hypervisor** is connected to the **Memory Controller** and **Memory Fabric**. The **System MMIO & Hypervisor** is also connected to the **System Control Backbone** and the **System MMIO & Hypervisor** at the bottom.

Network Wiring on NTU Golf Cart



Network Wiring on NTU Golf Cart

